LinkedIn Backend API Project Document By Aman

Project Link :- https://github.com/Amanshakya-code/LinkedIn-Clone

☕ Core Language & Framework

- **Java:** The primary programming language used for all backend logic.
- **Spring Boot:** The core framework used to rapidly build and bootstrap the REST APIs. It provides the embedded Tomcat server and auto-configuration.
- **Spring Web:** Used to build the RESTful APIs and handle HTTP requests/responses (using annotations like `@RestController`, `@GetMapping`, etc.).

🕸️ Microservices Architecture & Spring Cloud

- **Spring Cloud Netflix Eureka:** Used for **Service Discovery**. It acts as the registry where all microservices (User, Post, Chat, etc.) register themselves so they can find each other dynamically without hardcoded IP addresses.
- **Spring Cloud Gateway:** Acts as the **API Gateway**. It routes all incoming frontend traffic to the correct underlying microservice and handles global cross-cutting concerns (like validating JWTs before letting traffic inside).
- **Spring Cloud OpenFeign:** A declarative REST client used for **Inter-Service Communication**. It allows one microservice (like Job Service) to easily call another (like File Service) using simple Java interfaces instead of writing bulky `RestTemplate` code.

🔐 Security & Identity

- **Spring Security:** Used to secure the application, manage CORS, and protect the endpoints from unauthorized access.
- **JWT (JSON Web Tokens):** Used for **Stateless Authentication**. It securely transmits user identity between the client and the microservices without needing a centralized session database.
- **BCrypt (via Spring Security):** Used for cryptographically hashing user passwords before saving them to the database in the Auth Service.

🗄️ Database & Data Access

- **Spring Data JPA:** Used as the Data Access layer. It drastically reduces boilerplate code by allowing you to interact with the database using Java interfaces (`JpaRepository`) instead of writing raw SQL queries.
- **Hibernate:** The underlying ORM (Object-Relational Mapping) tool used by Spring Data JPA to map your Java entity classes (e.g., `@Entity User`) directly to database tables.
- **Relational Database (MySQL / PostgreSQL):** Based on the use of JPA and entities, the system is designed to run on a standard SQL relational database.

💬 Real-Time Communication

- **Spring WebSockets:** Used specifically in the `chat-service` to establish a persistent, bidirectional connection between users.
- **STOMP (Simple Text Oriented Messaging Protocol):** Used on top of WebSockets to format the messages and handle topic subscriptions (so User A's message gets routed perfectly to User B's screen).

🐳 DevOps, Build & Tooling

- **Docker:** Every microservice contains a `Dockerfile`. This is used for **Containerization**, allowing you to package the Java application and its environment into an isolated container that runs exactly the same way on any server.
- **Maven:** The build automation tool and dependency manager used across the project (evidenced by the `pom.xml` files).
- **Lombok:** A Java library that automatically plugs into your editor and build tools to auto-generate boilerplate code like Getters, Setters, and Constructors (often used in DTOs and Entities).

📘 LinkedIn Clone Microservices: Architecture & API Deep Dive

🏗️ Architectural Overview

This application is built using a **Microservices Architecture** with Spring Boot.

- **Why it's necessary:** A monolith would become too large and difficult to scale. By separating domains (Users, Posts, Jobs), we can scale the `post-service` (which has high read/write traffic) independently of the `company-job-service`.
- **What we used:** Spring Cloud (Eureka for Service Discovery, API Gateway for routing), Feign Clients for inter-service communication, and a shared File Service to centralize media management.

1. 🔐 Authentication Service (`auth-service`)

Purpose: Handles user identity, registration, and token generation. It acts as the gatekeeper for the application.

Endpoints Detail

- `POST /auth/register`
  - **Why we use it:** To onboard new users into the system and securely store their credentials.
  - **Why it's necessary:** We must separate raw passwords from the rest of the application. This endpoint encrypts the password (usually via BCrypt) before saving it to the database.
- `POST /auth/login`
  - **Why we use it:** To authenticate a user and return a JSON Web Token (JWT).
  - **Why it's necessary:** In a microservices architecture, stateful sessions (saving session IDs on the server) are hard to scale. JWTs allow **stateless authentication**, meaning any microservice can verify the user's identity just by looking at the token without hitting a central database.

🎙 Interview Questions: Auth Service

1. **Q: Why use JWT over Session Cookies in this microservices architecture?**
   - *A:* Microservices are distributed. If we used sessions, we would need a centralized session store (like Redis) so every service knows who is logged in. JWT is self-contained; it carries the user's ID and roles cryptographically signed. The API Gateway or individual services can validate it instantly without database lookups.
2. **Q: How do you handle password security in the `/auth/register` endpoint?**
   - *A:* Never store plain text. Use a hashing algorithm like BCrypt with a "salt" to defend against rainbow table attacks.
3. **Q: If a user is deleted in the `users-service`, how does the `auth-service` know?**
   - *A:* This requires distributed data management. We can use an event-driven approach (e.g., Kafka/RabbitMQ) where `users-service` publishes a "UserDeleted" event, and `auth-service` consumes it to delete the credentials.

2. 👤 User Service (`users-service`)

Purpose: Manages user profiles, work experiences, resumes, and the social graph (connections/friends).

Endpoints Detail

- `GET /api/users/profile` & `PUT /api/users`
  - **Why we use it:** To fetch and update the authenticated user's personal details (headline, about, location).
  - **Why it's necessary:** LinkedIn is a profile-centric app. Separating `UserInfo` from the `User` (auth) entity ensures that we don't load heavy profile data when we just need to authenticate someone.
- `GET /api/users/friends`
  - **Why we use it:** To get a list of a user's connections.
  - **Why it's necessary:** Needed for the feed algorithm and privacy checks (e.g., "Only connections can see this post").
- `POST /api/users/experiences` **(and CRUD counterparts)**
  - **Why we use it:** To build the user's resume/timeline.
  - **Why it's necessary:** Experiences are an array of complex objects. Creating a dedicated `experiences` table with a Many-to-One relationship to the User normalizes the database, preventing data duplication and allowing easy sorting by date.

🎙 Interview Questions: User Service

1. **Q: Why use DTOs (Data Transfer Objects) instead of returning the User entity directly from the Controller?**
   - *A:* Security and flexibility. If we return the entity, we might accidentally leak the password hash or database ID. DTOs allow us to shape the exact JSON structure the frontend needs, independent of how the database is structured.
2. **Q: How does the `users-service` communicate with the `file-service` to upload a profile picture?**
   - *A:* Using **OpenFeign** (Feign Clients). The User service takes the Multipart file from the user, calls the File Service via Feign, receives a URL back, and saves that URL in the `USER_FILES` database table.

3. 📝 Post Service (`post-service`)

Purpose: Manages the social feed, user posts, comments, and likes. This is typically the most heavily trafficked service.

Endpoints Detail

- `POST /api/posts` & `GET /api/posts`
  - **Why we use it:** To create text/media posts and retrieve the feed.
  - **Why it's necessary:** The core of user engagement. Paginating the `GET` request is strictly necessary here to prevent the database from crashing when trying to load thousands of posts at once.
- `POST /api/posts/comments` & `GET /api/posts/comments/replies/{id}`
  - **Why we use it:** To foster engagement via threaded conversations.
  - **Why it's necessary:** Implementing replies requires a self-referencing table structure (a comment can have a `parent_comment_id`). This allows for infinite nesting of replies (like Reddit or LinkedIn).
- `POST /api/posts/likes`
  - **Why we use it:** To toggle likes.
  - **Why it's necessary:** Likes are high-frequency actions. In a highly scalable system, these might actually be pushed to an in-memory datastore (Redis) first, then batch-inserted into the SQL database to reduce database write-locks.

🎙️ Interview Questions: Post Service

1. **Q: What is the N+1 Query Problem, and how might it occur when fetching Posts with their Comments?**
   - *A:* If we fetch 10 posts (1 query), and then for *each* post we run a query to fetch its comments (10 queries), we made 11 queries. If we fetch 100 posts, it's 101 queries. We fix this using `JOIN FETCH` in Hibernate/JPA to grab posts and comments in a single query.
2. **Q: When someone likes a post, how do you handle concurrency if 1000 people like it at the exact same millisecond?**
   - *A:* Instead of locking the database row to increment a `like_count`, we store individual records in a `POST_LIKES` table mapping `user_id` to `post_id`. To get the count, we use an aggregate function (`SELECT COUNT`), or maintain a cache in Redis that handles atomic increments (`INCR`), syncing with the DB periodically.

4. 🏢 Company & Job Service (`company-job-service`)

Purpose: The B2B domain. Manages company pages, job postings, and job applications.

Endpoints Detail

- `POST /api/company` & `POST /api/company/locations`
  - **Why we use it:** To allow users to create organization pages.
  - **Why it's necessary:** Companies have a One-to-Many relationship with locations (e.g., Google has offices in NY, SF, London). Separating locations into a different table allows location-based job searching.
- `POST /api/jobs` & `GET /api/jobs/sorted/{sortType}`
  - **Why we use it:** To post jobs and allow candidates to search them based on recency.
  - **Why it's necessary:** Proper indexing on the `posted_at` column in the database is necessary here to make the `sorted` endpoint fast.
- `POST /api/jobs/applied`
  - **Why we use it:** To link a User to a Job via an application.
  - **Why it's necessary:** This requires a classic Many-to-Many resolution table (`APPLIED_JOBS`). It tracks the exact timestamp of the application and the specific resume URL used (since users might update their master resume later, we need to snapshot the one they applied with).

Interview Questions: Company & Job Service

1. **Q: If a company is deleted, what happens to all the jobs posted by that company?**
   - *A:* This requires understanding **Cascading Deletes**. In the JPA entity, we would use `CascadeType.REMOVE` or handle it manually to ensure we don't end up with "orphan" jobs pointing to a non-existent `company_id`.
2. **Q: The Job Service needs the User's Resume from the User Service to apply for a job. How do you handle this?**
   - *A:* The Job service can call the User Service via a Feign Client to verify the resume exists and grab its URL. Alternatively, the client (frontend) passes the resume URL directly in the `CreateAppliedJobDto` to decouple the backend services.

---

5. Chat Service (`chat-service`)

Purpose: Handles 1-on-1 direct messaging between users.

Endpoints Detail

- `POST /api/chats` & `GET /api/chats/{id}`
  - **Why we use it:** To establish a connection "room" between two users.
  - **Why it's necessary:** We group messages by a `chat_id` rather than querying "all messages between User A and User B" every time, which drastically speeds up database retrieval.
- `POST /api/chats/messages`
  - **Why we use it:** To save text or media messages to the database.
  - **Why it's necessary:** While WebSockets handle real-time delivery, we *must* persist this data via standard HTTP POSTs (or via WebSocket STOMP handlers) so chat history isn't lost when the user refreshes the page.

🎙 Interview Questions: Chat Service

1. **Q: How do you achieve real-time messaging in Spring Boot?**
   - *A:* Using the `spring-boot-starter-websocket` dependency. We configure STOMP (Simple Text Oriented Messaging Protocol) endpoints. Clients subscribe to a topic (e.g., `/user/{id}/queue/messages`), and the server pushes messages instantly without the client needing to refresh or "poll" the server.
2. **Q: WebSockets maintain a persistent connection. In a microservice architecture with 5 Chat Service instances, how do you route a message to a user connected to a *different* instance?**
   - *A:* A single instance only knows about the WebSockets connected to it. To fix this, we introduce a **Message Broker** (like RabbitMQ or Redis Pub/Sub). When User A sends a message on Instance 1, Instance 1 publishes it to the broker. Instance 2 receives it from the broker and pushes it down the WebSocket to User B.

6. 📁 File Service (`file-service`)

Purpose: A centralized utility service for handling all media (images, PDFs, videos) across the platform.

Endpoints Detail

- `POST /api/files`
  - **Why we use it:** Takes a `MultipartFile` and uploads it to a storage bucket (like AWS S3, Cloudinary, or local storage).
  - **Why it's necessary:** Storing binary files (BLOBs) directly in a SQL database is a terrible practice—it makes the DB bloated, slow to back up, and expensive. We store files externally and only save the string URL in our SQL databases.
- `POST /api/files/batch-delete`
  - **Why we use it:** To clean up multiple files at once.
  - **Why it's necessary:** If a user deletes a Post that has 5 images attached, the `post-service` will call this endpoint to wipe all 5 images from the cloud storage in one network request, saving time and cloud costs.

Interview Questions: File Service

1. **Q: Why separate file uploading into its own microservice instead of letting the User Service and Post Service handle their own files?**
   - *A:* **Separation of Concerns (DRY Principle).** If we change our cloud provider from AWS S3 to Google Cloud Storage, we only have to rewrite the code in *one* service (`file-service`), not across the entire application.
2. **Q: How do you handle a user trying to upload a 5GB video that crashes your server memory?**
   - *A:* Implement file size limits in `application.properties` (`spring.servlet.multipart.max-file-size`). Additionally, use streaming uploads directly to the cloud provider (like S3 Pre-signed URLs) so the heavy file bypasses our Spring Boot backend entirely.

🌐 General Microservices Interview Questions (Bonus)

1. **Q: What is the role of the API Gateway in this project?**
   - *A:* It is the single entry point for the frontend frontend. It handles cross-cutting concerns: it validates the JWT securely via the `JwtAuthenticationFilter`, and then routes the traffic to the correct underlying microservice based on the URL path (e.g., `/api/users/**` goes to the user service).
2. **Q: How does the API Gateway know the IP addresses of the microservices?**
   - *A:* Through **Eureka Discovery Server**. When a service (like `post-service`) starts up, it registers itself with Eureka. The Gateway asks Eureka, "Where is the post-service?", and Eureka provides the IP address dynamically.
3. **Q: What happens if the `users-service` crashes, but the `post-service` tries to call it to get a user's name for a post?**
   - *A:* This can cause cascading failures. We should implement **Circuit Breakers** (like Resilience4j). If the user service fails repeatedly, the circuit breaker "opens" and the post service stops trying, instead returning a fallback response (e.g., displaying "Unknown User" instead of crashing the whole feed).

Part 1: High-Level Interview Questions & Architectural Justifications

1. Microservices Architecture

Q: Why did you choose a Microservices architecture instead of a Monolith for this LinkedIn clone?

- **The Answer & The "Why":** A monolith tightly couples all features together. In a social network like LinkedIn, the `Post Service` (Feed) will experience massively more read/write traffic than the `User Service` (Profile updates) or `Company Service`. By splitting them into microservices, we can **scale them independently**. If the feed goes viral, we can spin up 10 instances of the `post-service` while keeping only 2 instances of the `auth-service`. It also isolates failures—if the Chat service goes down, the rest of the application (Feed, Jobs) remains fully functional.

Q: What is the role of the API Gateway in your project? Why not let the frontend call microservices directly?

- **The Answer & The "Why":** If the frontend calls services directly, it has to manage CORS for multiple domains, track dynamic IP addresses, and handle authentication for every single request. The **API Gateway** acts as a single point of entry. It routes requests (e.g., `/api/users/**` goes to the user-service), centralizes cross-cutting concerns like validating JWT tokens, and hides the internal IP structure of the backend from the outside world.

Q: How do your microservices communicate with each other?

- **The Answer & The "Why":** They use **OpenFeign (Feign Clients)** for synchronous communication. For example, when a user applies for a job in the `company-job-service`, it might need to fetch the user's resume URL from the `users-service`. We use Feign because it declaratively maps HTTP calls to Java interfaces, making inter-service communication look like simple local method calls.

Q: What is Eureka Service Discovery, and why is it necessary?

- **The Answer & The "Why":** In a cloud environment, IP addresses of microservices change constantly (e.g., when they restart or autoscale). Hardcoding IPs in the API Gateway or Feign Clients would break the app. **Eureka** acts as a dynamic phonebook. Services register themselves with Eureka on startup. When the API Gateway needs to route to the `post-service`, it asks Eureka for the current, active IP address of that service.

2. Authentication & Security

Q: Why did you use JWT (JSON Web Tokens) instead of traditional Session IDs?

- **The Answer & The "Why":** Traditional sessions store state on the server (usually in memory). In a distributed microservices environment, if a user logs in to `Auth Service Instance A`, and their next request hits `User Service Instance B`, Instance B won't know they are logged in unless we build a complex, centralized session database (like Redis). **JWT is stateless.** The server signs the user's ID and roles cryptographically. Any microservice can independently verify the token's mathematical signature without querying a central database, making it highly scalable.

3. Database & Entity Management

Q: I see you have separate entity packages for each service. Do they share a single database, or use the "Database-per-Service" pattern?

- **The Answer & The "Why":** In strict microservices, you use **Database-per-Service**. The `users-service` has its own DB, and the `post-service` has its own DB. We do this to ensure loose coupling. If the Post team wants to switch from PostgreSQL to MongoDB (because NoSQL is often better for massive social feeds), they can do so without breaking the User team's relational database.

Q: How do you handle distributed data? For example, a Post belongs to a User. How do you link them if they are in different databases?

- **The Answer & The "Why":** The `Post` table simply stores the `user_id` as an integer/long. It **does not** have a physical Foreign Key constraint to the `User` table because they are in different databases. When the frontend needs to render a post with the user's name and avatar, we either aggregate this data on the frontend, use API Composition in the API Gateway, or have the `post-service` fetch the minimal user data via a Feign Client to return a combined DTO.

4. Real-time Features (Chat Service)

Q: Why did you use WebSockets for the Chat Service instead of standard HTTP REST APIs?

- **The Answer & The "Why":** HTTP is unidirectional and request-driven—the client must constantly "poll" (ask) the server if there are new messages, which wastes massive amounts of bandwidth and server CPU. **WebSockets** provide a persistent, bidirectional, full-duplex connection. When User A sends a message, the server instantly pushes it down the open pipe to User B without User B having to ask.

5. File Handling

Q: Why do you have a dedicated `file-service` instead of saving images directly in the database as BLOBs?

- **The Answer & The "Why":** Databases are optimized for structured data and indexing, not for storing massive binary files. Storing files in the DB causes database bloat, slows down backups, and consumes expensive database RAM. We delegate files to cloud storage (like AWS S3) or a dedicated file server, and only store the **URL string** in our SQL databases. Having a dedicated `file-service` centralizes the upload logic, adhering to the Single Responsibility Principle.

Part 2: Design Patterns Used in the Project

Your project heavily utilizes several standard software and cloud design patterns. Here is a detailed breakdown of each:

1. API Gateway Pattern

- **Description:** Instead of the frontend calling 10 different microservices via 10 different URLs, it calls one single server (the API Gateway). The Gateway acts as a reverse proxy, taking the request and routing it to the appropriate internal service.
- **Why it's used:** It encapsulates the internal system architecture. It also provides a centralized place for **cross-cutting concerns**. In your project, the `JwtAuthenticationFilter` lives in the Gateway. This means individual microservices don't have to write their own JWT parsing logic; the Gateway parses it, verifies it, and passes the safe `user_id` down to the microservices via HTTP Headers.

2. Service Registry / Discovery Pattern (Eureka)

- **Description:** A pattern where a central component (the registry) maintains the network locations of all running service instances.
- **Why it's used:** It enables client-side load balancing and dynamic routing. Without this pattern, microservices cannot find each other dynamically in a cloud environment where IP addresses are ephemeral.

3. Database-per-Service Pattern

- **Description:** Each microservice manages its own domain data exclusively. No service is allowed to directly query the database of another service; they must communicate via APIs.
- **Why it's used:** Ensures absolute loose coupling. If the `post-service` schema changes, it does not crash the `company-job-service`. It also allows polyglot persistence (using different database technologies tailored to specific service needs).

4. DTO (Data Transfer Object) Pattern

- **Description:** An object that is used to encapsulate data, and send it from one subsystem of an application to another (from the backend to the frontend).
- **Why it's used in your project:** You have classes like `CreatePostDto`, `UserDto`, etc. We use DTOs because we **never** want to expose our database Entities (`@Entity` classes) directly to the web.
  - *Security:* A `User` entity contains a password hash. A `UserDto` does not.
  - *Decoupling:* If the database table column changes from `first_name` to `f_name`, we can keep the `UserDto` property as `firstName` so we don't break the frontend UI.

5. Dependency Injection (DI) / Inversion of Control (IoC) Pattern

- **Description:** A core pattern of the Spring Framework. Instead of a class instantiating its dependencies (e.g., `UserService userService = new UserServiceImpl();`), the framework injects the dependencies via the constructor.
- **Why it's used:** It promotes loose coupling and makes unit testing incredibly easy (you can pass mock repositories into the constructor during testing). Your controllers use Constructor Injection for Services, and Services use Constructor Injection for Repositories.

6. Singleton Pattern (Spring Beans)

- **Description:** Ensures a class has only one instance and provides a global point of access to it.
- **Why it's used:** By default, all Spring `@Service`, `@Repository`, and `@RestController` classes are instantiated as Singletons by the Spring IoC container. You do not want to create a new instance of `UserServiceImpl` for every single HTTP request; one shared, stateless instance handles all traffic, saving massive amounts of memory.

7. Facade Pattern (Controllers)

- **Description:** Provides a simplified interface to a larger body of code.
- **Why it's used:** Your REST Controllers act as Facades. The frontend hits `/api/posts`, and the Controller simply calls `postService.createPost()`. The frontend doesn't need to know that `createPost()` involves saving to a DB, uploading an image via Feign, and triggering a notification. The complex business logic is hidden behind the simple Controller endpoint.

8. Proxy Pattern (Spring Data JPA & Security)

- **Description:** A class functioning as an interface to something else.
- **Why it's used: * JPA Repositories:** You write interfaces like `UserRepository extends JpaRepository`. You never write the implementation. Spring uses the Proxy pattern at runtime to generate the SQL queries and implement the interface for you.
  - **Transactions:** When you use `@Transactional`, Spring wraps your service method in a Proxy that opens a database connection before your method starts, and commits/rollbacks when your method finishes.

PART 1: Advanced Interview Questions & "The Why"

When interviewing for a backend or full-stack role with this project on your resume, interviewers will test your understanding of *distributed systems* and *architectural tradeoffs*.

1. Architectural & Microservices Questions

Q: Why did you choose a Microservices architecture instead of a Monolith for this project?

- **The Answer & The "Why":** A monolith tightly couples all features together. In a social network like LinkedIn, the `post-service` (the feed) handles massive read/write traffic compared to the `company-job-service`. By splitting them, we can **scale independently**. If a post goes viral, we can deploy 10 instances of the `post-service` while keeping only 2 instances of the `auth-service`. It also provides **fault isolation**—if the Chat service crashes, users can still apply for jobs and scroll their feed.

Q: What is the exact role of the API Gateway in your architecture? Why not let the frontend call microservices directly?

- **The Answer & The "Why":** If the React/Angular frontend calls services directly, it has to manage CORS for multiple ports, track dynamic IP addresses, and handle authentication headers for every single request. The **API Gateway** acts as a single point of entry (Reverse Proxy). It routes requests (`/api/users/**` goes to the user-service) and centralizes **cross-cutting concerns**. For example, the Gateway handles JWT validation, stripping the token and passing only the secure `userId` to the downstream microservices.

Q: Explain Eureka Service Discovery. Why couldn't you just hardcode the URLs of your microservices in the Gateway?

- **The Answer & The "Why":** In a cloud environment (like AWS or Kubernetes), IP addresses of microservices change constantly due to autoscaling, deployments, or server crashes. **Eureka acts as a dynamic phonebook.** When `users-service` starts, it registers its current IP with Eureka. When the Gateway needs to route a request to the User Service, it asks Eureka for the live IP address. This enables dynamic routing and client-side load balancing.

Q: How do your microservices communicate with each other?

- **The Answer & The "Why":** I used **OpenFeign (Feign Clients)** for synchronous REST communication. For example, when applying for a job, the `company-job-service` might need to verify a user's resume URL from the `file-service`. Feign is used because it abstract away the boilerplate of `RestTemplate` or `WebClient` into clean, declarative Java interfaces.

2. Security & Database Questions

Q: Why did you use JWT (JSON Web Tokens) instead of traditional HTTP Sessions?

- **The Answer & The "Why":** Traditional sessions store login state in the server's RAM. In a distributed environment, if a user logs into Instance A, but their next request hits Instance B, Instance B won't know they are logged in. **JWT is stateless.** The `auth-service` mathematically signs the token. Any other service (or the Gateway) can independently verify that signature without doing a database lookup or sharing memory, making the system infinitely scalable.

Q: Do your microservices share a single database, or do they have their own?

- **The Answer & The "Why":** They use the **Database-per-Service** pattern. The `users-service` has its own DB, and the `post-service` has its own DB. We do this to ensure absolute loose coupling. If the Post team wants to switch from PostgreSQL to MongoDB (which is often better for massive social feeds), they can do so without breaking the User team's SQL database.

3. Real-Time & Media Questions

Q: Why did you use WebSockets for the Chat Service instead of standard REST APIs?

- **The Answer & The "Why":** Standard HTTP is unidirectional—the client has to constantly "poll" or ask the server if there are new messages, which wastes massive amounts of bandwidth and overloads the server. **WebSockets** create a persistent, full-duplex connection. When User A sends a message, the server instantly pushes it down the open pipe to User B without User B ever asking for it.

PART 3: Entire Project Summary (Detailed)

You can use this summary for your GitHub Readme, Resume, or as your "Elevator Pitch" when an interviewer asks, "Tell me about your LinkedIn Clone project."

📝 Project Overview

The **LinkedIn Clone** is a highly scalable, distributed B2B social networking platform built using a **Spring Boot Microservices Architecture**. It replicates the core functionalities of a professional networking site, including user profile management, real-time social feeds, company page administration, job posting/application workflows, and real-time direct messaging.

The system is designed with enterprise-level scalability in mind, separating domains into isolated microservices to ensure fault tolerance, independent scaling, and polyglot persistence.

🏗️ System Architecture

The backend is composed of **8 distinct components**, communicating via synchronous REST calls (OpenFeign) and routed through a central Gateway.

1. **API Gateway:** The single entry point for all client requests. It implements a global `JwtAuthenticationFilter` to intercept requests, validate JWT signatures, and forward the authenticated `userId` to downstream services, ensuring internal services don't have to duplicate auth logic.

2. **Eureka Discovery Server:** Acts as the service registry. All microservices register their dynamic IP addresses here, allowing the API Gateway and Feign Clients to locate services without hardcoded URLs.
3. **Auth Service:** Responsible for identity management. It handles user registration (with BCrypt password hashing) and login, generating stateless JWTs for secure access.
4. **User Service:** Manages the professional identity. It handles CRUD operations for user profiles, work experiences, resumes, and maintains the social graph (connections/friends).
5. **Post Service:** The highest-traffic domain. It manages the social feed, allowing users to create text/media posts, leave threaded comments, and like content.
6. **Company & Job Service:** The B2B domain of the platform. It allows users to create organizational pages, manage office locations, post job listings, and process incoming job applications (linking a User's resume to a Job).
7. **Chat Service:** Enables real-time, 1-on-1 direct messaging between connections. It utilizes **WebSockets (STOMP)** for low-latency, full-duplex communication while persisting chat history in the database.
8. **File Service:** A centralized utility service that handles `MultipartFile` uploads (profile pictures, company logos, resumes, post images). It stores the raw binary data externally and returns accessible URLs to the other microservices, keeping the primary relational databases lightweight.

🔄 Example Request Flow: Applying for a Job

To understand how the microservices interact, consider the "Apply for Job" workflow:

1. The frontend sends a `POST` request to `/api/jobs/applied` through the **API Gateway**.
2. The Gateway validates the user's JWT, extracts their `userId`, and routes the request to the **Company & Job Service**.
3. The Job Service receives the request. If the user attached a new resume, the Job Service uses a **Feign Client** to call the **File Service** to upload the PDF.
4. The File Service returns the secure file URL.
5. The Job Service saves the application record in its database (linking the `job_id`, `user_id`, and `resume_url`) and returns a success response to the client.

🚀 Technical Highlights & Best Practices

- **Stateless Security:** By leveraging JWTs, the system avoids the overhead of distributed session management (like Redis session replication).
- **Separation of Concerns:** Strict adherence to the Database-per-Service pattern ensures that a failure or schema change in the `post-service` has zero impact on the `users-service`.
- **Data Masking:** Extensive use of the **DTO Pattern** prevents over-posting attacks and ensures sensitive entity data is never leaked to the client.
- **Centralized Exception Handling:** Implementation of `@RestControllerAdvice` ensures that all microservices return consistent, standardized JSON error responses (e.g., `ErrorDetails` DTO) to the frontend.